

Contents

1 1. Overview

Workwarrior is a profile-based shell productivity system that wraps five open-source tools — TaskWarrior, TimeWarrior, JRNL, Hledger, and Bugwarrior — into a single unified command surface. The system provides:

- **Profile isolation** — each profile is an independent directory containing all tool data for one work context
- **Unified CLI** — a single `ww` dispatcher routes all commands to service scripts
- **Natural language input** — 627 compiled heuristic regex rules translate plain English to tool commands before optionally falling back to a local or remote LLM
- **Browser UI** — a locally-served web interface (Python 3 stdlib only) with 15+ panels, real-time SSE updates, and a unified command input
- **GitHub sync** — two complementary engines for pulling issues from 20+ services and bidirectional sync between individual tasks and GitHub issues
- **Extensible service architecture** — new services are executable scripts in `services/<category>/`, discovered at runtime with no registration step

Entry point: `bin/ww` (main CLI dispatcher)

Shell bootstrap: `bin/ww-init.sh` (sourced by `.bashrc`/`.zshrc` at shell start)

Canonical install path: `~/ww`

2 2. Architecture

User types: `p-work`

- `lib/shell-integration.sh` sets `TASKRC`, `TASKDATA`, `TIMEWARRIORDB`, etc.
- all tools now operate on the work profile's data

User types: `task add "Ship API" due:friday`

- TaskWarrior writes to `profiles/work/.task/`
- on-modify hook starts TimeWarrior tracking

User types: `ww browser`

- Python3 HTTP server starts on `localhost:7777`
- serves static UI + REST API + SSE events
- reads profile data via same env vars

User types in browser CMD: `"add task review code and start tracking"`

- HeuristicEngine matches against 627 rules
- splits compound command on `"and"`

→ executes: task add review code + timew start review code

2.1 Data Flow Summary

Shell init

- ww-init.sh sourced
 - shell-integration.sh defines p-<name> aliases and shell functions
 - ww alias registered

Profile activation (p-work)

- use_task_profile("work") called
- 5 env vars exported
- all subsequent tool calls use profile data

ww <command> [args]

- bin/ww validates profile active (or allow-listed command)
- discovers service script in services/<command>/
- profile-level service checked first (overrides global)
- executes service with remaining args

ww browser

- services/browser/server.py starts ThreadingHTTPServer on :7777
- SSE channel open for real-time profile events
- REST API handles panel data and CMD execution
- CMD input → HeuristicEngine → 627 rules → AI fallback → ww subcommand execution

3 3. Directory Structure

bin/

ww	Main CLI dispatcher (all ww commands route here)
ww-init.sh	Shell bootstrap (sourced at shell start)
profile	Profile activation helper
custom	Custom service launcher
export	Export launcher
x	Delete launcher

lib/

core-utils.sh	Core bash libraries (sourced, not executed directly)
profile-manager.sh	WW_BASE resolution, profile validation, last-profile state
shell-integration.sh	Profile lifecycle: create, delete, backup, import, restore
logging.sh	Shell function injection, alias management, rc file writes
github-api.sh	log_info/log_success/log_warning/log_error/log_step
github-sync-state.sh	GitHub REST API via gh CLI
sync-pull.sh	Sync state persistence (JSON per task)
	GitHub → TaskWarrior pull

sync-push.sh	TaskWarrior → GitHub push
sync-bidirectional.sh	Orchestrates pull+push with conflict window
sync-detector.sh	Change detection (task side + GitHub side)
field-mapper.sh	Field mapping between TaskWarrior and GitHub formats
conflict-resolver.sh	Last-write-wins conflict resolution with configurable window
annotation-sync.sh	TaskWarrior annotations GitHub comments sync
sync-permissions.sh	Per-UDA sync permission tokens
config-loader.sh	Profile config loading (jrnل.yaml, ledgers.yaml, bugwarrior)
config-utils.sh	YAML parsing utilities
taskwarrior-api.sh	tw_get_task, tw_update_task, tw_get_field wrappers
bugwarrior-integration.sh	Bugwarrior config and pull helpers
dependency-installer.sh	Platform-aware tool installer with version cards
installer-utils.sh	Install path resolution, shell RC block management
export-utils.sh	Profile data export (JSON, CSV, markdown)
delete-utils.sh	Profile deletion with backup safety
profile-stats.sh	Profile statistics and reporting
shortcode-registry.sh	Shortcut/alias registry read/write
error-handler.sh	GitHub error classification
services/	Service scripts (executable, discovered at runtime)
profile/	
create-ww-profile.sh	Full profile creation (dirs, taskrc, hooks, aliases)
manage-profiles.sh	list/info/delete/backup/import/restore
profile-tool.sh	Profile info display
urgency.sh	ww profile urgency - urgency coefficient management
subservices/	
profile-uda.sh	ww profile uda - full UDA management surface
profile-density.sh	ww profile density - TWDensity integration
uda-manager.sh	Interactive UDA CRUD
custom/	
configure-journals.sh	ww custom journals
configure-ledgers.sh	ww custom ledgers
configure-tasks.sh	ww custom tasks
configure-times.sh	ww custom times
configure-issues.sh	ww custom issues / ww issues custom
github-sync.sh	ww issues push/pull/sync/enable/disable/status
questions/	
q.sh	Main questions dispatcher
handlers/	Per-service question handlers
templates/	YAML question templates
extensions/	
extensions.sh	ww extensions taskwarrior list/search/info
taskwarrior.py	Extension data fetcher
find/	
find.sh	ww find dispatcher
find.py	Python search implementation

groups/	
groups.sh	ww group list/show/create/add/remove/delete
models/	
models.sh	ww model list/providers/add/set-default
export/	
export.sh	ww export (JSON, CSV, markdown)
browser/	
server.py	Python3 HTTP server - 15+ panels, SSE, CMD, heuristic engine
static/app.js	Browser UI - sidebar, task editor, all panels
static/index.html	HTML shell - dark terminal aesthetic
static/style.css	Styles
cmd/	
cmd.log	JSONL log of all CMD submissions
ctrl/	
ctrl.sh	ww ctrl status/ai-mode/prompt-ww/prompt-ai
x-delete/	
x.sh	ww x - profile/data deletion with safety backups
servers/	Server management
scripts/	Utility scripts
find/	Cross-profile search
weapons/	
gun/	taskgun passthrough - bulk task series generator
sword/	Task splitting into sequential subtasks with dependencies
scripts/	
compile-heuristics.py	Heuristic compilation
scan-taskwarrior-extensions.py	Extension registry scanner
functions/	Shell helper functions (sourced at init)
journals/	Journal helpers
ledgers/	Ledger helpers
tasks/	TaskWarrior helpers and extensions
times/	TimeWarrior helpers
issues/	Issue tracking helpers
tools/	
list/list.py	List management tool (ww list)
config/	
ai.yaml	AI mode, access points, preferred provider
cmd-heuristics.yaml	Compiled NL→command rules (627 rules)
cmd-heuristics-corpus.yaml	Synthetic corpus for heuristic validation
models.yaml	LLM provider/model registry
ctrl.yaml	CTRL service settings
groups.yaml	Profile group definitions

shortcuts.yaml	Shortcut definitions
profiles/ <name>/ .taskrc .task/ .timewarrior/ journals/ ledgers/ jrn1.yaml ledgers.yaml .config/	User profiles (created at runtime, gitignored) TaskWarrior config (UDAs, reports, urgency) TaskWarrior database TimeWarrior database Journal files Hledger ledger files Journal name → file mapping Ledger name → file mapping Service configs (bugwarrior)
system/ CLAUDE.md TASKS.md fragility-register.md gates/ roles/ plans/ specs/ audits/ workflows/	Control plane (not shipped to users) Primary agent context document Canonical task board File fragility classifications Gate contract templates Agent role definitions Planning documents Specification documents Audit outputs Process workflows

4 4. Environment Variables

Set automatically by `use_task_profile()` when a profile is activated via `p-<name>`:

Variable	Value	Used By
WARRIOR_PROFILE	Active profile name (e.g. <code>work</code>)	All ww scripts
WORKWARRIOR_BASE	Profile base directory (<code>~/ww/profiles/work</code>)	All ww scripts
TASKRC	Path to profile <code>.taskrc</code>	TaskWarrior
TASKDATA	Path to profile <code>.task/</code>	TaskWarrior
TIMEWARRIORDB	Path to profile <code>.timewarrior/</code>	TimeWarrior

Rule: Nothing in the system hardcodes paths. All path resolution goes through

these variables. A lib function or service that constructs a path by concatenating `~/ww/profiles/<name>/...` is a bug — it must use `$WORKWARRIOR_BASE`.

5 5. Profile Model

5.1 Structure

<code>profiles/<name>/</code>	
<code>.taskrc</code>	TaskWarrior config (UDAs, reports, urgency coefficients, hooks)
<code>.task/</code>	TaskWarrior database (taskchampion.sqlite3 + supporting files)
<code>.timewarrior/</code>	TimeWarrior database (data/ + .timewarrior config)
<code>journals/</code>	JRNL journal files
<code>ledgers/</code>	Hledger ledger files
<code>jrnl.yaml</code>	Journal name → file path mapping
<code>ledgers.yaml</code>	Ledger name → file path mapping
<code>.config/</code>	Per-profile service configs
<code> bugwarrior/</code>	Bugwarrior config (bugwarriorrc or bugwarrior.toml)
<code> taskcheck/</code>	Taskcheck config
<code>services/</code>	Per-profile service overrides (shadow global services)
<code>ai.yaml</code>	Per-profile AI mode override (optional)

5.2 Lifecycle

<code>ww profile create <name></code>	<i># Creates directory structure, writes .taskrc, injects hooks</i>
<code>ww profile list</code>	<i># Lists all profiles with last-active timestamps</i>
<code>ww profile info <name></code>	<i># Shows profile details, resource inventory, sync state</i>
<code>ww profile delete <name></code>	<i># Marks deleted, creates safety backup</i>
<code>ww profile backup <name></code>	<i># Archives to profiles/<name>-backup-<timestamp>.tar.gz</i>
<code>ww profile import <archive></code>	<i># Creates new profile from archive</i>
<code>ww profile restore <archive></code>	<i># Replaces existing profile from archive</i>
<code>ww remove <name></code>	<i># Full removal: scrubs aliases, removes config references, o</i>

5.3 Multiple Named Resources

A profile can have multiple named journals and ledgers:

<code>ww journal add <name></code>	<i># Creates journals/<name>.txt and adds mapping to jrnl.yaml</i>
<code>ww journal list</code>	<i># Lists all journals in active profile</i>
<code>ww journal remove <name></code>	<i># Removes journal file and mapping</i>
<code>ww journal rename <old> <new></code>	<i># Renames journal file and updates mapping</i>
<code>ww ledger add <name></code>	<i># Creates ledgers/<name>.journal and adds mapping to ledgers</i>
<code>ww ledger list</code>	<i># Lists all ledgers in active profile</i>

5.4 Profile Groups

```
ww group create <name>           # Create a named group
ww group add <group> <profile>    # Add profile to group
ww group list                     # List all groups
ww group show <name>              # Show group membership
ww group remove <group> <profile> # Remove profile from group
```

Groups enable batch operations: `ww export --group work` exports all profiles in the work group.

5.5 Backup/Restore Policy

Profile backup creates a `.tar.gz` archive of the entire profile directory. The archive is self-contained — `ww profile import <archive>` reconstructs the profile including all data, config, and resource mappings.

Safety backups are created automatically before `profile delete` and `ww remove`. The backup path is displayed on screen before deletion proceeds.

6. Shell Bootstrap

6.1 bin/ww-init.sh

Sourced by the user's `.bashrc` or `.zshrc` at shell start. Responsibilities:

1. Sources `lib/shell-integration.sh`
2. Calls `ensure_shell_functions()` to verify injection is up to date
3. Restores the last active profile if `WARRIOR_PROFILE` is unset

6.2 Shell Functions Injected

Function	Description
<code>task [args]</code>	TaskWarrior with active profile's TASKRC and TASKDATA
<code>timew [args]</code>	TimeWarrior with active profile's TIMEWARRIORDB
<code>j [name] "text"</code>	JRNL entry — optional journal name, falls back to default
<code>l [name] <cmd></code>	Hledger command — optional ledger name, falls back to default
<code>i [args]</code>	Bugwarrior pull shorthand
<code>q <template></code>	Questions template runner
<code>list [args]</code>	Quick task list (<code>ww list passthrough</code>)

Function	Description
<code>search <term></code>	Cross-profile search
<code>p-<name></code>	Activate named profile — created per profile

6.3 RC File Management

`shell-integration.sh` manages a `# WW ALIASES` section in all detected shell rc files (`.bashrc`, `.zshrc`). Profile aliases are written to this section. `remove_profile_aliases()` removes only the entries for the specified profile. The section is idempotent — re-running never duplicates entries.

7 7. CLI Dispatcher

7.1 bin/ww

The main entry point for all `ww` commands. Approximately 709 lines. Responsibilities:

1. Sources all `lib/*.sh` files
2. Checks profile active state (or allow-listed command flag)
3. Parses first argument as the service category
4. Discovers service script: checks `$WORKWARRIOR_BASE/services/<category>/` first, then `services/<category>/`
5. Validates known subcommands at the routing level for security-sensitive services
6. Executes the service with remaining arguments

7.2 Service Discovery

The dispatcher scans for executables matching the service category. Profile-level services shadow global services of the same name. If no match is found, the dispatcher prints a list of available services and exits 1.

7.3 Allow-Listed Commands

Some commands run without an active profile: `profile create`, `profile list`, `deps install`, `deps check`, `help`, `version`. All others require `WARRIOR_PROFILE` to be set.

7.4 Security Constraint

For the browser CMD endpoint, `ALLOWED_SUBCOMMANDS` is a frozenset of valid service names. POST `/cmd` requests must have a first token matching this set.

No `sh -c`, no eval. Unknown subcommands return HTTP 400.

8 8. Core Libraries

8.1 lib/core-utils.sh

- `resolve_ww_base()` — resolves `WORKWARRIOR_BASE` from `WARRIOR_PROFILE` if not set
- `validate_profile_active()` — exits 1 with message if no profile is active
- `get_last_profile()` / `set_last_profile()` — persists last-used profile name to `~/.ww_last_profile`
- `profile_exists(name)` — returns 0 if profile directory exists

8.2 lib/profile-manager.sh

- `create_profile(name)` — full profile creation: `mkdir` structure, write `.taskrc` template, inject TaskWarrior hooks, create `jrnl.yaml` and `ledgers.yaml`, write bugwarrior config template, call `create_profile_aliases()`
- `delete_profile(name)` — creates safety backup, removes profile directory
- `backup_profile(name)` — archives to `profiles/<name>-backup-<timestamp>.tar.gz`
- `import_profile(archive)` — extracts archive to `profiles/`, creates aliases
- `restore_profile(archive)` — replaces existing profile directory from archive

8.3 lib/shell-integration.sh

See Section 6.

- `use_task_profile(name)` — core activation: exports 5 env vars, sets last profile
- `create_profile_aliases(name)` — writes `p-<name>` to all rc files
- `remove_profile_aliases(name)` — removes all aliases for profile from rc files
- `get_ww_rc_files()` — returns list of shell rc files to write to
- `add_alias_to_section(name, value, [file])` — idempotent alias injection
- `ensure_shell_functions()` — verifies `ww-init.sh` source line present in all rc files

Re-source guard: `[[ -n "${SHELL_INTEGRATION_LOADED:-}" ]] && return 0`. Never `readonly` — normal user re-sourcing must not error.

8.4 lib/logging.sh

All user-facing output goes through these functions. Never use raw `echo` in lib or services.

	Function	Output	Exit
	<code>log_info "msg"</code>	<code>[INFO] msg</code> to stdout	—
	<code>log_success "msg"</code>	<code>[] msg</code> to stdout	—
	<code>log_warning "msg"</code>	<code>[WARN] msg</code> to stderr	—
	<code>log_error "msg"</code>	<code>[ERROR] msg</code> to stderr	—
	<code>log_step "msg"</code>	<code>[+] msg</code> to stdout	—

8.5 lib/taskwarrior-api.sh

Wrappers for TaskWarrior operations that avoid direct `task` calls in lib functions (keeping testability):

- `tw_get_task(uuid)` — returns task JSON
- `tw_update_task(uuid, mods)` — applies modifications
- `tw_get_field(uuid, field)` — returns single field value

8.6 lib/config-utils.sh

YAML parsing utilities for `jrnل.yaml`, `ledgers.yaml`, and `ai.yaml`:

- `yaml_get(file, key)` — reads a single scalar value
- `yaml_get_mapping(file)` — returns key:value pairs from a mapping
- `yaml_set(file, key, value)` — writes or updates a value

8.7 lib/config-loader.sh

Loads and validates GitHub sync configuration from bugwarrior config:

- `init_github_sync_config()` — main entry point, locates config, loads, validates
- `load_github_sync_config(config_path)` — exports `GITHUB_LOGIN`, `GITHUB_USERNAME`, `GITHUB_TOKEN`, `GITHUB_PROJECT_TEMPLATE`
- `validate_github_sync_config()` — checks required fields, returns 1 with specific error if missing
- Oracle token pattern: `@oracle:eval:gh auth token` — evaluated at load time via `gh auth token`

9 9. Service Architecture

9.1 Contract

Every service script must:

1. Start with `#!/usr/bin/env bash` and `set -euo pipefail`
2. Respond to `--help` / `-h` with a one-line description and usage example
3. Use exit codes: 0 success, 1 user error, 2 system/internal error
4. Log via `lib/logging.sh` functions — never raw `echo` for user-facing messages
5. Not write to profile directories directly — call lib functions
6. Be discoverable: placed at `services/<category>/` with executable permission

9.2 Minimal Service Template

```
#!/usr/bin/env bash
set -euo pipefail

source "$WORKWARRIOR_BASE/lib/logging.sh"

USAGE="ww myservice - one-line description
Usage: ww myservice <subcommand> [args]
"

case "${1:-}" in
  --help|-h) echo "$USAGE"; exit 0 ;;
  list)      # implementation ;;
  info)
    [[ -z "${2:-}" ]] && { log_error "info requires a name"; exit 1; }
    # implementation ;;
  *)
    log_error "Unknown subcommand: '${1:-}'"
    echo "$USAGE"
    exit 1
  ;;
esac
```

9.3 Profile-Level Service Override

Services at `profiles/<name>/services/<category>/` shadow global services. The dispatcher checks the profile path first. This enables per-profile service customizations without modifying global services.

9.4 Service Tiers

Tier	Description	Use Case
1 — Simple	Single-file, direct tool calls	Single-tool wrappers, simple data reads
2 — Compound	Multiple subcommands, uses lib functions	Lifecycle services (create/list/delete/backup)
3 — Complex	State management, external APIs, background processes	GitHub sync, browser server

10 10. Service Registry

Domain	Commands	Description
<code>ww profile</code>	create, list, info, delete, backup, import, restore, uda, urgency, density	Profile lifecycle and configuration
<code>ww journal</code>	add, list, remove, rename	Named journal management
<code>ww ledger</code>	add, list, remove, rename	Named ledger management
<code>ww group</code>	list, create, show, add, remove, delete	Profile groups for batch operations
<code>ww model</code>	list, providers, show, add-provider, remove-provider, add-model, set-default, env, check	LLM provider and model registry
<code>ww ctrl</code>	status, ai-on, ai-off, ai-status, ai-mode, prompt-ww, prompt-ai	AI mode, prompt settings, UI config
<code>ww find</code>	<code><term></code> with filters	Cross-profile search across tasks, time, journals, ledgers
<code>ww issues</code>	sync, push, pull, status, enable, disable, custom, uda	GitHub two-way sync + Bugwarrior pull
<code>ww custom</code>	journals, ledgers, tasks, times, issues	Interactive configuration wizards
<code>ww extensions</code>	taskwarrior list/search/info/refresh	Extension registry
<code>ww export</code>	JSON, CSV, markdown	Profile data export
<code>ww questions</code>	list, new, delete, <code><template></code>	Template-based capture workflows

Domain	Commands	Description
<code>ww browser</code>	start, stop, status, export	Locally-served web UI
<code>ww remove</code>	<profile>, -keep, -all, -archive-all, -delete-all	Profile removal with scrubbing
<code>ww shortcut</code>	list, info, add, remove	Shortcut/alias reference
<code>ww deps</code>	install, check	Dependency management
<code>ww compile-heuristics</code>	-verbose, -digest	Recompile NL→command rules
<code>ww gun</code>	<args>	Bulk task series (wraps taskgun)
<code>ww sword</code>	<task> -p N [-interval Nd]	Task splitting with dependency chains
<code>ww next</code>	—	CFS-inspired next-task recommendation
<code>ww schedule</code>	—	Auto-scheduler (wraps taskcheck)
<code>ww mcp</code>	install, status	MCP server for AI agent access
<code>ww tui</code>	install	taskwarrior-tui installer
<code>ww list</code>	—	List management tool

11. Browser UI

11.1 Overview

`ww browser` starts a locally-served web interface on `http://localhost:7777`. No npm, no build step, no cloud, no accounts. Python 3 stdlib only.

```

ww browser                # Start on port 7777, open browser
ww browser --port 9090    # Custom port
ww browser --no-open      # Start without opening browser
ww browser stop           # Stop server
ww browser status         # Show running state

```

11.2 Server Implementation

- `services/browser/server.py` — single Python file implementing `ThreadingHTTPServer`
- Static files served from `services/browser/static/` on each request (no caching — changes visible on refresh)
- `ThreadingHTTPServer` handles SSE connections (which hold sockets open) without blocking concurrent POST requests

- SSE endpoint: `GET /events` — streams profile change events as `text/event-stream`

11.3 REST API

	Method	Path	Description
GET		/health	Liveness check with profile name and version
GET		/data/tasks	Pending tasks for active profile
GET		/data/time	Time intervals and totals
GET		/data/journal	Recent journal entries (active journal)
GET		/data/ledger	Account balances and recent transactions
GET		/data/ctrl	AI settings and resolved provider
GET		/data/models	LLM provider registry
GET		/data/profile	Active profile info and resource lists
GET		/data/groups	Profile group definitions
GET		/data/sync	GitHub sync state for active profile
GET		/data/questions	Available question templates
POST		/cmd	Execute a ww subcommand
POST		/profile/switch	Switch active profile
GET		/events	SSE stream

11.4 Security Model

`ALLOWED_SUBCOMMANDS` frozenset: every `POST /cmd` request is validated — the first token must appear in the frozenset. Unknown subcommands return HTTP 400. No `sh -c`, no eval.

This bounds the attack surface: XSS in the UI can only invoke known ww subcommands. Known ww subcommands cannot exec arbitrary shell commands.

11.5 Panels

Panel	Data	Actions
Tasks	Full task list with UDAs	Inline edit, start/stop/done, add, annotate
Time	Today's total, weekly breakdown, recent intervals	Start, stop
Journals	Entry list with expand/collapse	New entry, journal selector dropdown
Ledgers	Account balances, recent transactions, income statement	New transaction, ledger selector

Panel	Data	Actions
CMD	Natural language + direct command input	Route indicator /
CTRL	AI mode toggle	off / local-only / local+remote
Models	LLM provider and model registry	Add provider, set default
Groups	Profile group management	Create, add member
Sync	GitHub sync dashboard	Sync, push, status
Questions	Template browser	Run template
Profile	Profile info and resource management	Switch profile, resource list
Weapons bar	Gun, Sword, Next, Schedule icons	Opens weapon panel

11.6 CMD Input Routing

Input received

- Compound check: contains "and"/"then"/"also"/"plus"?
- Yes: split into segments
- Each segment:
 - `HeuristicEngine.match(segment, threshold=0.8)`
 - If match: execute
 - If no match: AI fallback if configured
 - If no AI: return error with suggestion
- Collect results, return to UI

12 12. Heuristic Engine

12.1 Overview

627 compiled regex rules across 19 command domains. No network. No latency. Loaded at server startup from `config/cmd-heuristics.yaml`.

12.2 Pipeline

1. **Compound split** — input containing “and”, “then”, “also”, “plus” is split into segments

2. **Rule evaluation** — each segment tested against all 627 rules, highest confidence wins
3. **Threshold check** — match must exceed 0.8 confidence to accept
4. **AI fallback** — if no rule matches, input goes to configured LLM (if enabled)
5. **Execution** — matched commands executed against active profile

12.3 Rule Structure

Each rule contains: - **pattern** — compiled regex - **action_template** — output command template with capture group references - **confidence** — score (0.0–1.0) - **domain** — one of 19 command domains

12.4 Confidence Levels by Phrasing Variation

Variation	Confidence	Example
Passthrough	1.0	task add review budget
Imperative	0.95	add a task to review the budget
Declarative	0.90	I need a task for reviewing the budget
Interrogative	0.90	can you create a task to review the budget
Shorthand	0.90	task: review budget due friday
Verbose	0.85	I would like to add a new task for reviewing the budget

12.5 Command Domains

task · time · journal · ledger · profile · group · model · ctrl · service · issues · find · schedule · gun · next · mcp · browser · extensions · custom · shortcut

12.6 Date Expression Normalization

Input	Output
“tomorrow”	due:tomorrow

Input	Output
"next week"	due:1w
"friday"	due:friday
"end of month"	due:eom
"in 3 days"	due:3d

12.7 Compiler

scripts/compile-heuristics.py generates config/cmd-heuristics.yaml:

```
ww compile-heuristics           # Standard run
ww compile-heuristics --verbose  # Every rule with test results
ww compile-heuristics --digest   # + CMD log analysis + new rules from AI hits
```

Compiler reads: bin/ww case branches, config/shortcuts.yaml, config/command-syntax.yaml, and optionally services/cmd/cmd.log.

Steps: 1. Scan command sources for command definitions 2. Generate regex patterns for each phrasing variation 3. Validate against synthetic corpus (config/cmd-heuristics-corpus.yaml) 4. Resolve conflicts (higher-confidence rule wins) 5. Fill coverage gaps 6. Write output

12.8 Self-Improvement Loop

Every CMD submission is logged to services/cmd/cmd.log as JSONL: {input, route, output, success, timestamp}.

--digest flag reads this log, finds successful AI translations that had no heuristic match, generates new rules from those translations. Run ww compile-heuristics --digest regularly to expand heuristic coverage from real usage.

13. AI Integration

13.1 Configuration

```
# config/ai.yaml
mode: off           # off / local-only / local+remote
preferred_provider: ollama
access_points:
  cmd_ai: true      # Enable AI in CMD service
```

Per-profile override: profiles/<name>/ai.yaml with same structure. Profile config takes precedence over global.

13.2 Modes

Mode	Behavior
off	Heuristics only. No AI calls. Unmatched inputs return error.
local-only	Heuristics first, then ollama if no match. No data leaves machine.
local+remote	Heuristics → local LLM → remote provider fallback chain.

13.3 Provider Registry

```
# config/models.yaml
providers:
  - name: ollama
    type: ollama
    base_url: http://localhost:11434
  - name: openai
    type: openai
    api_key_env: OPENAI_API_KEY
models:
  default: llama3.2
  fallback_chain: [llama3.2, gpt-4o-mini]
```

CLI management:

```
ww model add-provider ollama ollama http://localhost:11434
ww model add-provider openai openai
ww model set-default llama3.2
ww model list
ww model check           # Test connectivity to all configured providers
```

13.4 Runtime Control

```
ww ctrl ai-on           # Enable (uses mode from config)
ww ctrl ai-off          # Disable
ww ctrl ai-status       # Show resolved state (global + profile + effective)
ww ctrl ai-mode local-only # Set mode
```

Browser CTRL panel mirrors all CLI controls. Changes take effect immediately
— no restart required.

14 14. GitHub Sync Engine

14.1 Two Engines

Bugwarrior — one-way pull from 20+ services into TaskWarrior.

ww github-sync — two-way bidirectional sync between individual tasks and GitHub issues.

These are complementary. Use both simultaneously.

14.2 Bugwarrior

Pulls issues from: GitHub, GitLab, Jira, Trello, Bitbucket, Taiga, Pagure, and 13+ more.

```
ww issues custom          # Interactive configuration wizard (per-profile)
i pull                    # Pull from all configured services
i status                  # Show sync state
```

Configuration stored in `profiles/<name>/config/bugwarrior/bugwarriorrc`.

Injected UDAs on created tasks: `githubissue`, `githuburl`, `githubrepo`, `githubauthor` — classified as `[github]` source in the UDA registry.

14.3 ww github-sync

Two-way sync between individual TaskWarrior tasks and GitHub issues.

```
ww issues enable <task_id> <issue_num> <org/repo>  # Link task to issue
ww issues disable <task_id>                        # Unlink
ww issues sync                                       # Two-way sync all linked tasks
ww issues push                                     # Push local changes to GitHub only
ww issues pull                                     # Pull GitHub changes only
ww issues status                                   # Show sync state for all linked tasks
```

14.4 Field Mapping

Bidirectional (both directions):

TaskWarrior	GitHub Issue
description	title (truncated to 256 chars on push)
status	state (pending/started → OPEN, completed/deleted → CLOSED)
priority	labels (H → priority:high, M → priority:medium, L → priority:low)
tags	labels (system tags excluded)

TaskWarrior	GitHub Issue
annotations	comments (with [tw] prefix to prevent loop)

Pull-only (GitHub → TaskWarrior):

	GitHub Issue	TaskWarrior UDA
Issue number		githubissue
Issue URL		githuburl
Repository		githubrepo
Author		githubauthor
createdAt		entry (first sync only)
closedAt		end (if status → completed)
updatedAt		modified

14.5 Sync Engine Files

All 10 files classified HIGH FRAGILITY. Changes require extended risk brief.

File	Role
lib/github-api.sh	GitHub REST API via gh CLI: get/update/label/comment
lib/github-sync-state.sh	Sync state persistence — JSON per task in <code>.config/ww-sync/</code>
lib/sync-detector.sh	Change detection — compares current state to last sync state
lib/sync-pull.sh	GitHub → TaskWarrior pull with orphan detection
lib/sync-push.sh	TaskWarrior → GitHub push with label management
lib/sync-bidirectional.sh	Orchestrates pull+push with conflict window management
lib/field-mapper.sh	Field mapping between TW and GitHub formats
lib/conflict-resolver.sh	Last-write-wins with configurable window
lib/annotation-sync.sh	TW annotations GitHub comments
services/custom/github-sync.sh	Service entry point and CLI dispatch

14.6 Conflict Resolution

Last-write-wins with a configurable conflict window (default: 60 seconds).

If both sides changed within the conflict window, the sync reports the conflict rather than silently overwriting. The conflict is surfaced in `ww issues status` with both values and timestamps.

Outside the conflict window: the more recently modified side wins.

14.7 Orphan Detection

If a linked GitHub issue is deleted, `sync_pull_issue()` receives `[not-found]` from the API. Rather than hard-erroring, it logs a warning and skips the task. The user is prompted to run `ww issues disable <uuid>` to clean up the link.

14.8 Annotation/Comment Sync

TaskWarrior annotations are pushed as GitHub comments prefixed with `[tw]`. GitHub comments prefixed with `[tw]` are skipped on pull (preventing sync loops). Non-prefixed GitHub comments are pulled as TaskWarrior annotations prefixed with `[github]`.

14.9 Oracle Token Pattern

Bugwarrior config stores: `github.token = @oracle:eval:gh auth token`

The oracle directive is evaluated at load time via `gh auth token`. The actual token is never written to the config file. `lib/config-loader.sh` detects and evaluates this pattern.

15. Weapons System

Weapons are service-level tools that manipulate profile data in special ways — creating, slicing, and packaging tasks. They appear as a weapons bar in the browser sidebar.

15.1 Sword

Native to ww. Splits a single task into N sequential subtasks with dependency chains.

```
ww sword <task_id> -p <N> # Split into N parts
ww sword <task_id> -p <N> --interval <Nd> # N-day intervals between parts
ww sword <task_id> -p <N> --prefix "Phase" # Custom prefix
```

Each subtask receives: - Description: “Part N of: {original description}” - Parent task’s project and tags - Due date: offset by $N \times \text{interval}$ from now - Dependency: on previous subtask in the chain

Parent task is archived after split. The chain is strictly sequential — a subtask cannot be completed until all its dependencies are complete.

15.2 Gun

Wraps taskgun (Rust). Bulk task series generator.

```
ww gun <args>    # Arguments passed through to taskgun
```

Respects active profile (reads TASKRC/TASKDATA from env). Returns taskgun output with error handling.

15.3 Next

Wraps the `next` binary. CFS-inspired next-task recommendation.

```
ww next          # Recommends single optimal next task
```

Factors: TaskWarrior urgency score, deadline proximity, context signals.

15.4 Schedule

Wraps taskcheck. Auto-scheduler for time blocks.

```
ww schedule
```

15.5 Planned Weapons

Weapon	Status
Bat	Planned
Fire	Planned
Slingshot	Planned

15.6 Weapon Constraints

All weapons: - Read TASKRC/TASKDATA from environment (profile isolation respected) - Work via `POST /cmd` in the browser UI - Never modify data outside active profile scope - Respond to `--help`

16 16. UDA System

16.1 Overview

TaskWarrior User Defined Attributes are first-class in Workwarrior. A profile can carry 100+ UDAs. The browser UI renders all UDAs in the task inline editor dynamically.

16.2 UDA Types

	Type	TaskWarrior declaration	Use
string	uda.<name>.type=string		Text values
numeric	uda.<name>.type=numeric		Numbers, billable hours
date	uda.<name>.type=date		Dates
duration	uda.<name>.type=duration		Estimated time

16.3 Source Classification

	Badge	Source
[github]		Injected by Bugwarrior from issue tracking services
[extension:<name>]		Added by TaskWarrior extensions or hooks
[custom]		Defined by user via <code>ww profile uda add</code>

Extension-classified UDAs are protected from accidental deletion.

	Extension	UDAs	Badge
TWDensity	density, densitywindow		[extension:twdensity]
taskcheck	estimated, time_map		[extension:taskcheck]

16.4 Management

```
ww profile uda list           # All UDAs with source badges
ww profile uda add <name>    # Interactive UDA creation wizard
ww profile uda remove <name> # Remove (blocked for extension UDAs)
ww profile uda group <name>  # Apply UDA template group
ww profile uda perm <name> nosync # Set sync permission token
ww profile uda perm <name> private # Mark as private (no AI access)
ww profile uda perm <name> noai  # Exclude from AI context
```

16.5 Sync Permissions

Permission tokens stored in `profiles/<name>/.config/ww-sync-permissions.yaml`:

	Token	Effect
<code>nosync</code>		Excluded from github-sync push
<code>private</code>		Excluded from AI context
<code>noai</code>		Excluded from AI context
<code>readonly</code>		Pull-only, never pushed

16.6 Urgency Tuning

```
ww profile urgency      # Interactive urgency coefficient tuner
```

Tunable coefficients: `urgency.due.coefficient`, `urgency.priority.coefficient`, `urgency.age.coefficient`, `urgency.project.coefficient`, `urgency.tags.coefficient`, per-UDA urgency weights.

Results written to profile's `.taskrc`. Per-profile tuning means different contexts can have different priority models.

16.7 Density Scoring

```
ww profile density      # TWDensity integration - due-date density scoring
```

TWDensity extension injects `density` and `densitywindow` UDAs. Higher density = more tasks due in the window = higher urgency.

17 17. Extensions

17.1 TaskWarrior Extensions Registry

```
ww extensions taskwarrior list      # Browse community extensions
ww extensions taskwarrior search <term>  # Search by name or tag
ww extensions taskwarrior info <name>    # Show details
ww extensions taskwarrior refresh      # Update registry from upstream
```

Registry populated by `scripts/scan-taskwarrior-extensions.py` — scans the TaskWarrior community extension list and indexes metadata.

17.2 Hook Events

	Event	When
on-launch	Every task	invocation
on-add	New task	creation
on-modify	Task field	change
on-exit	After task	command exits

Workwarrior's own integration uses `on-modify` to auto-start TimeWarrior when `task start` is called.

17.3 TimeWarrior Extensions

Python scripts that run against TimeWarrior data. Custom reports, summaries, integrations. Installed to `~/.timewarrior/extensions/`.

18. Questions System

Template-based structured capture workflows. A question template is a YAML file defining a series of prompts that write to tasks, journal, or ledger.

```
q <template>           # Run a template (interactive prompts)
ww questions list      # List available templates
ww questions new       # Create a new template interactively
ww questions delete    # Remove a template
```

Templates in `services/questions/templates/`. Per-profile templates in `profiles/<name>/templates/` shadow global templates.

18.1 Template Structure

```
name: standup
description: Daily standup notes
prompts:
  - field: yesterday
    label: "What did you do yesterday?"
    target: journal
    journal: standup
  - field: today
    label: "What will you do today?"
    target: task
    project: standup
  - field: blockers
    label: "Any blockers?"
    target: journal
    journal: standup
```

19 19. Export System

```
ww export json          # Full profile export as JSON
ww export csv           # Tasks as CSV
ww export markdown      # Tasks as Markdown table
ww export --group <g>  # Export all profiles in group
```

Export scopes: tasks only, time only, journals only, ledgers only, or all resources.

Implemented in lib/export-utils.sh. Output to stdout by default; --output <path> writes to file.

20 20. Fragility Register

Classification	Files	Constraints
HIGH FRAGILITY	lib/github-api-extended.sh, lib/github-sync-verify.sh, lib/sync-pull.sh, lib/sync-push.sh, lib/sync-bidirectional.sh, lib/field-mapper.sh, lib/sync-detector.sh, lib/conflict-resolver.sh, lib/annotation-sync.sh, services/custom/github-sync.sh	Extended risk brief required. Verifier sign-off before merge. Integration tests required.
SERIALIZED	bin/ww, lib/shell-integration.sh	One writer at a time. Changes affect all users immediately.
NEVER COMMIT	profiles/*/task-changes.sqlite3, profiles/*/config/, profiles/*/list/, *.sqlite3	Use task-changes.sqlite3, Never, in version control.

21. Shell Coding Standards

Every script in the project must follow these rules. Violations are Gate B failures.

```
#!/usr/bin/env bash
set -euo pipefail
```

- **Bash, not sh.** `#!/usr/bin/env bash` on every script. `#!/bin/sh` is not acceptable.
- **set -euo pipefail** is the second line. No exceptions.
 - `-e`: exit on error
 - `-u`: exit on undefined variable
 - `-o pipefail`: pipe failure propagates
- **Error propagation via return codes**, not exit traps or `exit 1` in lib functions.
- **Logging via lib/logging.sh** functions. Never `echo` for user-facing messages in lib or services.
- **Absolute paths always.** Use `$WORKWARRIOR_BASE/...`, never relative paths.
- **Quote all variable expansions:** `"$var"` not `$var`. `"${array[@]}"` not `${array[@]}`.
- **Functions in snake_case.** No camelCase. No kebab-case.
- **All local variables declared with local.**

- **No cd in lib functions.** Use full paths.
- **Exit codes:** 0 = success, 1 = user error, 2 = system/internal error.

21.1 Variable Naming

Scope	Convention
Global env vars	UPPER_SNAKE_CASE
Local function vars	lower_snake_case with local
Function names	lower_snake_case

21.2 Function Template

```
function_name() {
    local arg1="${1:?function_name requires arg1}"
    local arg2="${2:-default_value}"
    local result

    # implementation

    echo "$result"
    return 0
}
```

22 22. Testing

22.1 BATS Tests

```
bats tests/ # All BATS test suites
bats tests/test-models-service.bats # Specific suite
bats tests/test-profile-manager.bats
bats tests/test-shell-integration.bats
bats tests/test-github-sync.bats
```

BATS (Bash Automated Testing System) provides shell-native test assertions. Each test file covers one service or lib component.

22.2 Python Tests (pytest)

```
python3 -m pytest services/browser/ # Browser server and heuristic engine
```

Tests cover: HTTP endpoint behavior, heuristic rule matching, compound command splitting, AI fallback routing, SSE event emission.

22.3 Test Policy

- Tests run locally. No CI currently active.
 - Every implementation task (Gate C) requires test execution before merge.
 - Sync engine changes require integration tests against a test GitHub repository.
 - Shell function changes require BATS suite updates.
-

23 23. Install Policy

23.1 Core Toolchain

`ww deps install` is the canonical install path.

	Tool	Minimum Version	Package
TaskWarrior	3.0.0		<code>task</code>
TimeWarrior	1.4.0		<code>timew</code>
JRNL	4.0.0		<code>jrn1</code> (via <code>pipx</code>)
Hledger	1.30		<code>hledger</code>
Bugwarrior	1.22.0		<code>bugwarrior</code> (via <code>pipx</code>)
GitHub CLI	2.0.0		<code>gh</code>
Python 3	3.9		<code>python3</code>
pipx	any		<code>pipx</code>

23.2 Platform Matrix

	Platform	Manager	Auto-install
macOS		<code>brew</code>	Yes
Debian/Ubuntu		<code>apt</code>	No (emits command)
Fedora/RHEL		<code>dnf</code>	No (emits command)
Arch/Manjaro		<code>pacman</code>	No (emits command)
Other		—	Emits manual URLs

23.3 Install Steps

```
git clone <repo-url> ~/ww
cd ~/ww
./install.sh
source ~/.bashrc  # or source ~/.zshrc
ww deps check
ww profile create work
p-work
```

23.4 Optional Extensions

```
ww tui install      # taskwarrior-tui (full-screen TUI)
ww mcp install      # MCP server (AI agent access)
```

MCP install requires uv (Python package manager). Auto-installed on macOS via brew.

24 24. Dependency Table

Dependency	Type	Role	Upstream
TaskWarrior	Core tool	Task management, UDAs, hooks	taskwarrior.org
TimeWarrior	Core tool	Time tracking	timewarrior.net
JRNL	Core tool	Journal entries	jrnl.sh
Hledger	Core tool	Double-entry accounting	hledger.org
Bugwarrior	Core tool	Issue pull from 20+ services	github.com/GothenburgBitFactory/bugwarrior
GitHub CLI (gh)	Required	GitHub sync authentication and API	cli.github.com
Python 3 (3.9)	Required	Browser UI server	python.org
pipx	Required	JRNL and Bugwarrior isolation	pypa.github.io/pipx
taskgun	Weapon	Bulk task series (Gun weapon)	github.com/hamzamohdzubair/taskgun
taskcheck	Weapon	Auto-scheduler (Schedule weapon)	—
taskwarrior-tui	Optional	Full-screen TUI	github.com/kdheepak/taskwarrior-tui

Dependency	Type	Role	Upstream
taskwarrior-mcp	Optional	MCP server for AI agents	github.com/hnsstrk/taskwarrior- mcp
ollama	Optional AI	Local LLM provider	ollama.ai
TWDensity	Optional extension	Due-date density scoring	—

Workwarrior Technical Standard v0.1.0 — Babb — workwarrior@babb.tel